script.js

```
let audioCtx = null;
let gainNode = null;
let isAudioInitialized = false;

// 1. 온보딩 시작하기 버튼을 명확히 터치했을 때 호출되는 오디오 초기화 함수
function startLesson() {
    if (isAudioInitialized) return;

    try {
        const AudioContext = window.AudioContext || window.webkitAudioContext;
        audioCtx = new AudioContext();

        gainNode = audioCtx.createGain();
        // 🔊 초기 청취 가능성 상향: 방안에서 듣는 볼륨을 0.08에서 0.18로 올려 처음부터 잘 들리도록 조절
        gainNode.gain.setValueAtTime(0.18, audioCtx.currentTime);

        const bufferSize = 2 * audioCtx.sampleRate;
        const noiseBuffer = audioCtx.createBuffer(1, bufferSize, audioCtx.sampleRate);
        const output = noiseBuffer.getChannelData(0);

        for (let i = 0; i < bufferSize; i++) {
            output[i] = Math.random() * 2 - 1;
        }

        const whiteNoise = audioCtx.createBufferSource();
        whiteNoise.buffer = noiseBuffer;
        whiteNoise.loop = true;

        const filter = audioCtx.createBiquadFilter();
        filter.type = 'lowpass';
        filter.frequency.setValueAtTime(1100, audioCtx.currentTime); // 비 내리는 먹먹한 톤 보정

        whiteNoise.connect(filter);
        filter.connect(gainNode);
        gainNode.connect(audioCtx.destination);

        whiteNoise.start();
        isAudioInitialized = true;
        console.log("⚡ 웹 오디오 가동 완료: 처음부터 은은한 빗소리가 정상 작동합니다.");

        // UI에서 스플래시 오버레이 화면을 자연스럽게 제거
        const overlay = document.getElementById('startOverlay');
```

```
        overlay.style.opacity = '0';
        overlay.style.pointerEvents = 'none';

    } catch (e) {
        console.error("웹 오디오 API 초기화 실패:", e);
    }
}

// 2. 아파트 창문 커튼을 치면 합성 중인 빗소리가 웅장하게 커지는 메인 액티비티 로직
function triggerNode1() {
    const frame = document.getElementById('windowFrame');

    if (!frame.classList.contains('open')) {
        frame.classList.add('open');
        document.getElementById('rainBg').style.display = 'block';
        document.getElementById('targetSentence').classList.add('show');

        // 🔊 커튼 인터랙션: 커튼을 치면 0.18에서 0.65(65% 수준)로 소리가 확 쏟아지듯 부드럽게 증폭!
        if (gainNode && audioCtx) {
            gainNode.gain.linearRampToValueAtTime(0.65, audioCtx.currentTime + 0.25);
        }

        setTimeout(() => {
            document.getElementById('ccqPanel').classList.add('show');
        }, 1000);
    }
}

/**
 * [Node 1 -> Node 2] 스프린트 확장 스크립트
 * 기존 Web Audio 인프라(AudioContext 등)가 전역에 생성되어 있다고 가정합니다.
 */

// 오디오 파라미터 변수 확장 제어 레이어
let lowPassFilter; // 과거의 잔상을 표현할 로우패스 필터
let dripInterval;  // 물방울 신스 스케줄러

/**
 * 노드 1의 [Yes, right now!] 검증 버튼 클릭 시 호출되는 화면/오디오 전환 함수
 */
function finishDemo() {
    const node1 = document.getElementById('node-1');
    const node2 = document.getElementById('node-2');

    // 1. 시각적 페이드 아웃/인 시퀀스
```

```
    node1.style.opacity = '0';
    setTimeout(() => {
        node1.classList.add('is-hidden');

        node2.classList.remove('is-hidden');
        // 브라우저 렌더링 큐 반영 후 페이드인
        setTimeout(() => {
            node2.style.opacity = '1';
        }, 50);

        // 2. 청각적 시제 엣지 전환 실행 (Web Audio API 제어)
        transitionAudioToPast();
    }, 1200); // CSS transition 시간(1.2s)과 동기화
}

/**
 * Web Audio API 실시간 필터 및 노드 전환 로직
 * 웅장한 실시간 빗소리를 '둔탁한 과거의 잔상'으로 깎고, 간헐적 물방울 신스를 추가합니다.
 */
function transitionAudioToPast() {
    // 오디오 컨텍스트가 활성화되어 있지 않다면 안전하게 리턴
    if (!window.audioCtx || window.audioCtx.state === 'suspended') return;

    // 1. 기존 노드 1의 메인 Gain 노드 또는 오디오 아웃풋 체인 사이에 로우패스 필터 삽입
    // (기존 화이트노이즈/핑크노이즈 소스가 mainGain에 물려있다고 가정)
    if (!lowPassFilter) {
        lowPassFilter = window.audioCtx.createBiquadFilter();
        lowPassFilter.type = 'lowpass';

        // 기존 볼륨 노드(mainGain)와 최종 목적지(destination) 사이에 연결 구조 변경
        if (window.mainGain) {
            window.mainGain.disconnect();
            window.mainGain.connect(lowPassFilter);
            lowPassFilter.connect(window.audioCtx.destination);
        }
    }

    // 2. 오디오 파라미터 자동 자동화(Automation) 감쇄: 현재 시점부터 3초간 서서히 차단 주파수 변경
    const now = window.audioCtx.currentTime;

    // 주파수를 300Hz로 대폭 깎아서 빗소리를 완전히 먹먹하고 멀게 만듦 (과거의 잔상 톤)
    lowPassFilter.frequency.setValueAtTime(8000, now);
    lowPassFilter.frequency.exponentialRampToValueAtTime(300, now + 3.0);

    // 전체 볼륨도 과거의 기억처럼 아주 은은하게 감쇄
    if (window.mainGain) {
```

```
        window.mainGain.gain.setValueAtTime(window.mainGain.gain.value, now);
        window.mainGain.gain.linearRampToValueAtTime(0.08, now + 3.0);
    }

    // 3. 물방울 뚝-뚝 떨어지는 실시간 효과음 신디사이저(Drip Synth) 스케줄링 가동
    startDripSynth();
}

/**
 * Pure Web Audio API 기반의 오디오 합성: 물방울 소리 (Drip Synth)
 */
function playDripSound() {
    if (!window.audioCtx) return;

    const now = window.audioCtx.currentTime;

    // 오실레이터와 엔벨롭(Gain) 노드 생성
    const osc = window.audioCtx.createOscillator();
    const gainNode = window.audioCtx.createGain();

    osc.type = 'sine';

    // 인지언어학적 청각 매핑: 물방울이 떨어지며 튕기는 '틱'하는 고주파 변조 연출
    osc.frequency.setValueAtTime(600, now);
    osc.frequency.exponentialRampToValueAtTime(1200, now + 0.04); // 급격히 주파수를 올려 물방울 고유의 배음 합성

    // 볼륨 엔벨롭 설정 (매우 짧고 간결하게 툭 떨어지는 소리)
    gainNode.gain.setValueAtTime(0.0, now);
    gainNode.gain.linearRampToValueAtTime(0.15, now + 0.005);
    gainNode.gain.exponentialRampToValueAtTime(0.001, now + 0.08);

    // 노드 연결 및 플레이
    osc.connect(gainNode);
    gainNode.connect(window.audioCtx.destination);

    osc.start(now);
    osc.stop(now + 0.09);
}

/**
 * 노드 2의 주기적인 물방울 합성 엔진 구동
 */
function startDripSynth() {
    if (dripInterval) clearInterval(dripInterval);

    // 2초~4초 사이의 불규칙적 인터벌로 물방울 소리를 합성하여 리얼리티 부여
```

```
    dripInterval = setInterval(() => {
        playDripSound();
        // 물웅덩이 시각 요소에 가벼운 트리거 이펙트 연동 가능
        triggerPuddleRipple();
    }, Math.random() * 2000 + 2000);
}

function triggerPuddleRipple() {
    const ripple = document.querySelector('.puddle-ripple');
    if(ripple) {
        ripple.style.transform = 'scale(1.5)';
        ripple.style.opacity = '0.4';
        setTimeout(() => {
            ripple.style.transform = 'scale(0)';
            ripple.style.opacity = '0';
        }, 600);
    }
}

/* --- Node 2 교육 패널 CCQ 인터랙션 함수 --- */
function handleNode2Wrong() {
    const feedback = document.getElementById('node2-feedback');
    feedback.classList.remove('is-hidden');
    feedback.style.color = '#f25c54';
    feedback.innerHTML = "❌ Look outside/inside carefully! The rain has stopped now. But look at the water puddles on the ground!";
}

function handleNode2Correct() {
    const feedback = document.getElementById('node2-feedback');
    feedback.classList.remove('is-hidden');
    feedback.style.color = '#06d6a0';
    feedback.innerHTML = "🎯 Excellent! It is NOT raining right now. But the wet puddles prove that it <strong>was raining</strong> this morning. You grasped the Tense Edge!";

    // 정답 시 물방울 소리를 음악적인 화음으로 피드백 연출 가능
    playDripSound();
}

```